Documentación de un sistema CRUD con Spring Boot y JPA

Creador de Oportunidades: Juan José Gracia Gómez

Gestor del Conocimiento: William Alexander Matallana Porras

Ingeniería de Sistemas Y computación

Universidad de Cundinamarca

Chía

2026

Introducción:

En el presente documento se presenta el desarrollo de una API REST orientada a la gestión de diferentes entidades académicas, como usuarios, perfiles, cursos, categorías, libros y estudiantes. Esta API fue construida utilizando el framework Spring Boot, el cual permite desarrollar aplicaciones de manera más ágil y organizada dentro del entorno Java.

El proyecto se desarrolló siguiendo una arquitectura por capas, organizando el código en paquetes como controller, service, repository, model y configuration, lo cual facilita la estructura general del sistema y su desarrollo.

En cuanto a la persistencia de datos, se utilizó JPA (Java Persistence API) junto con Hibernate como proveedor, lo que permite realizar el mapeo entre las clases del sistema y las tablas de la base de datos. Para el almacenamiento de la información se empleó PostgreSQL como sistema gestor de base de datos.

Adicionalmente, se implementó Swagger OpenAPI como herramienta para la documentación y prueba de la API, permitiendo visualizar los endpoints y ejecutar solicitudes directamente desde el navegador.

Finalmente, el sistema se basa en la implementación de operaciones CRUD (Crear, Leer, Actualizar y Eliminar) para cada una de las entidades, lo cual permite gestionar la información de manera completa. Todo esto se desarrolló con base en el código implementado a lo largo del proyecto, integrando correctamente cada una de las capas.

Descripción del código del sistema:

El proyecto se desarrolló utilizando una arquitectura por capas, lo que permitió organizar el código de manera más clara y ordenada. Para ello, se estructuraron paquetes como controller, service, repository, model y configuration, los cuales en conjunto permiten separar responsabilidades dentro del sistema. Esta organización facilita la comprensión del proyecto, así como su mantenimiento y posible escalabilidad, ya que cada parte cumple un rol específico dentro del funcionamiento general de la API.

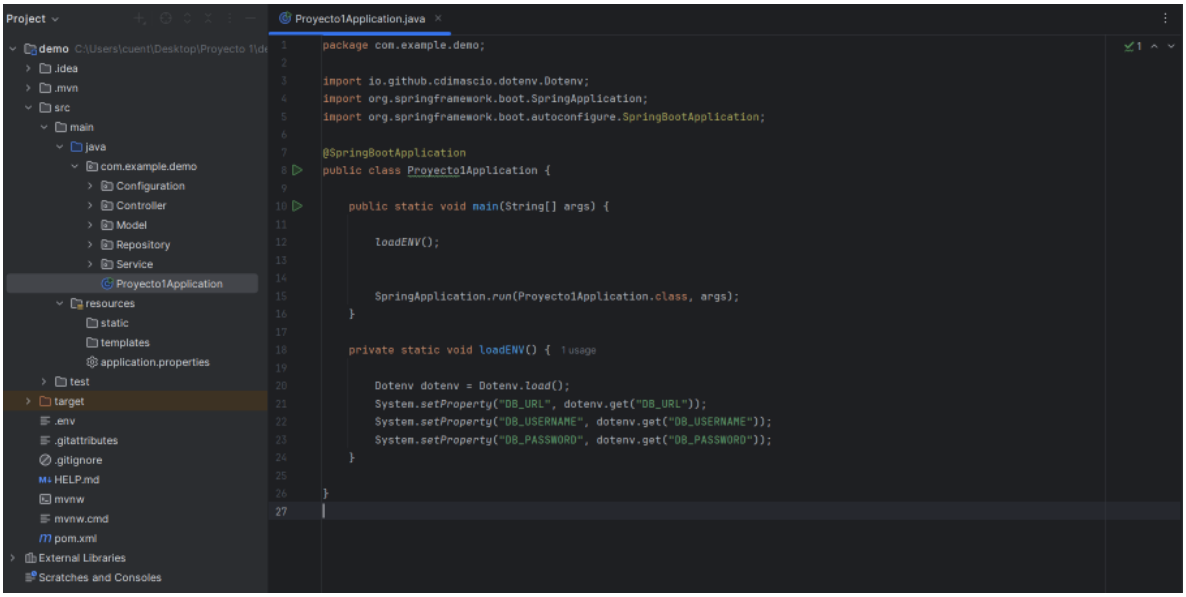


Imagen: Vista general del Proyecto.

Controller:

El package controller contiene las clases encargadas de manejar las solicitudes HTTP que llegan a la API. En esta capa se definen los endpoints que permiten realizar operaciones como crear, consultar, actualizar y eliminar información. Además, los controladores se comunican con la capa service para procesar las solicitudes y devuelven las respuestas al

cliente utilizando estructuras como ResponseEntity, incluyendo los códigos de estado HTTP correspondientes.

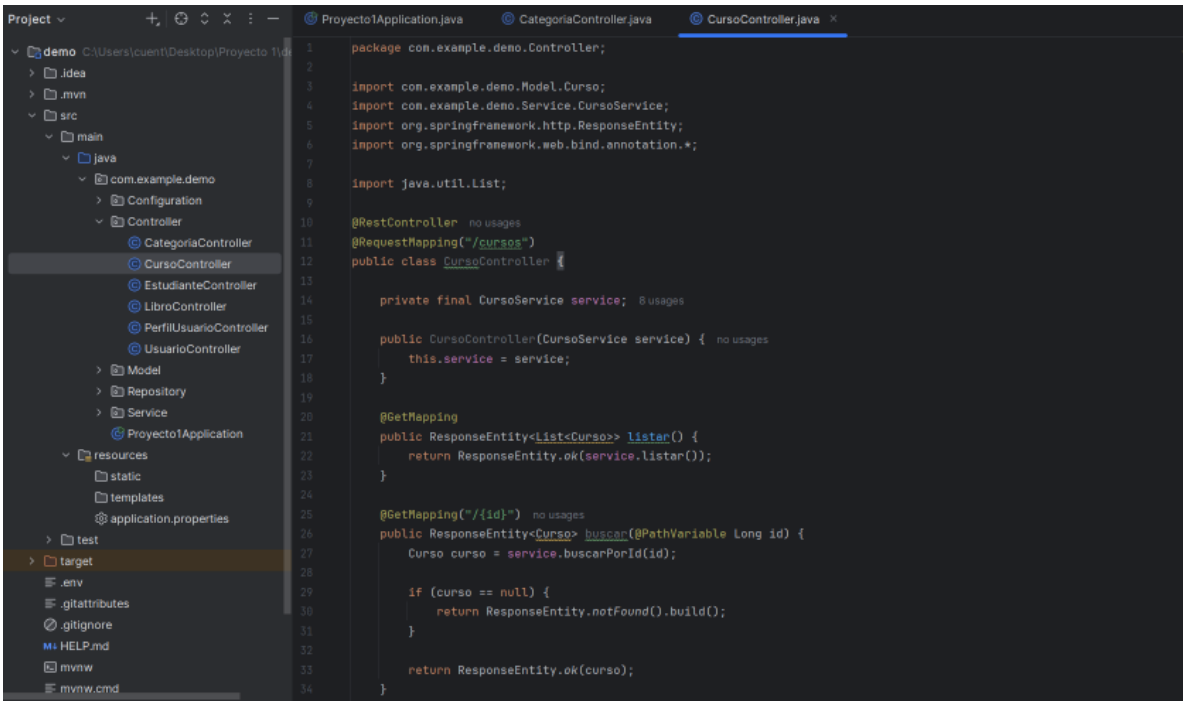


Imagen: Package Controller

Service:

El package controller contiene las clases encargadas de manejar las solicitudes HTTP que llegan a la API. En esta capa se definen los endpoints que permiten realizar operaciones como crear, consultar, actualizar y eliminar información. Además, los controladores se comunican con la capa service para procesar las solicitudes y devuelven las respuestas al cliente utilizando estructuras como ResponseEntity, incluyendo los códigos de estado HTTP correspondientes.

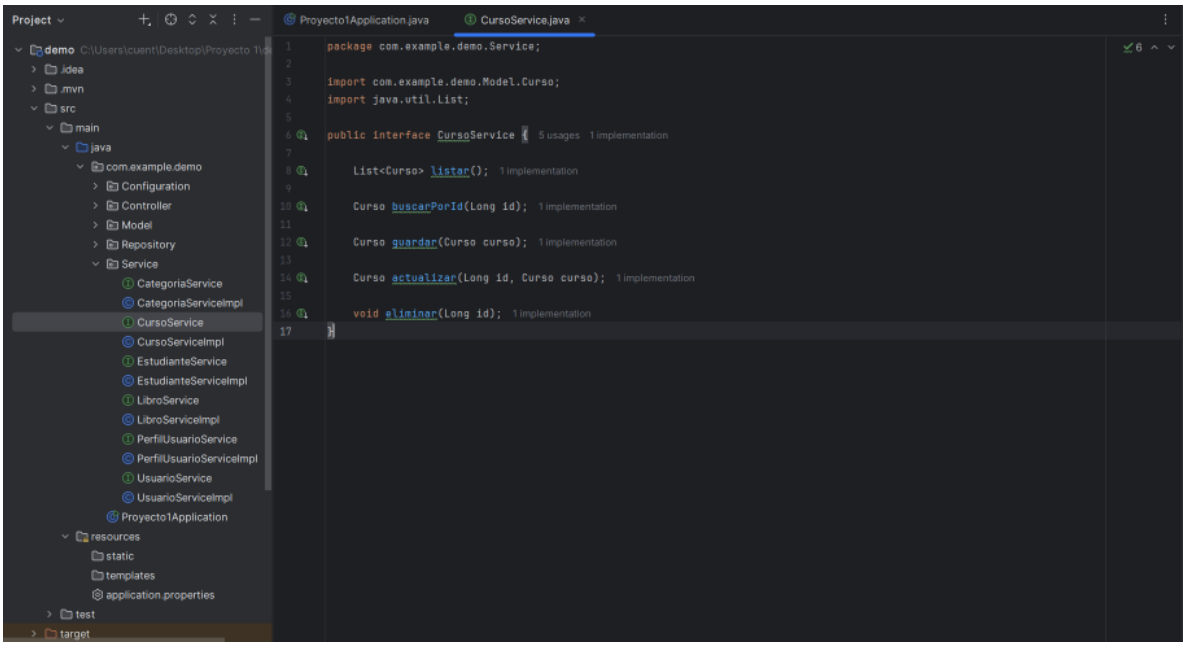


Imagen: Package Service

Repository:

El package repository se encarga de la comunicación directa con la base de datos. Cada repositorio extiende JpaRepository, lo que permite acceder a métodos ya definidos para realizar operaciones CRUD sin necesidad de escribir consultas SQL manualmente. A través de esta capa, se gestionan las entidades como Usuario, Curso, Libro, entre otras, permitiendo guardar, consultar, actualizar y eliminar información en la base de datos PostgreSQL de manera automática mediante JPA y Hibernate.

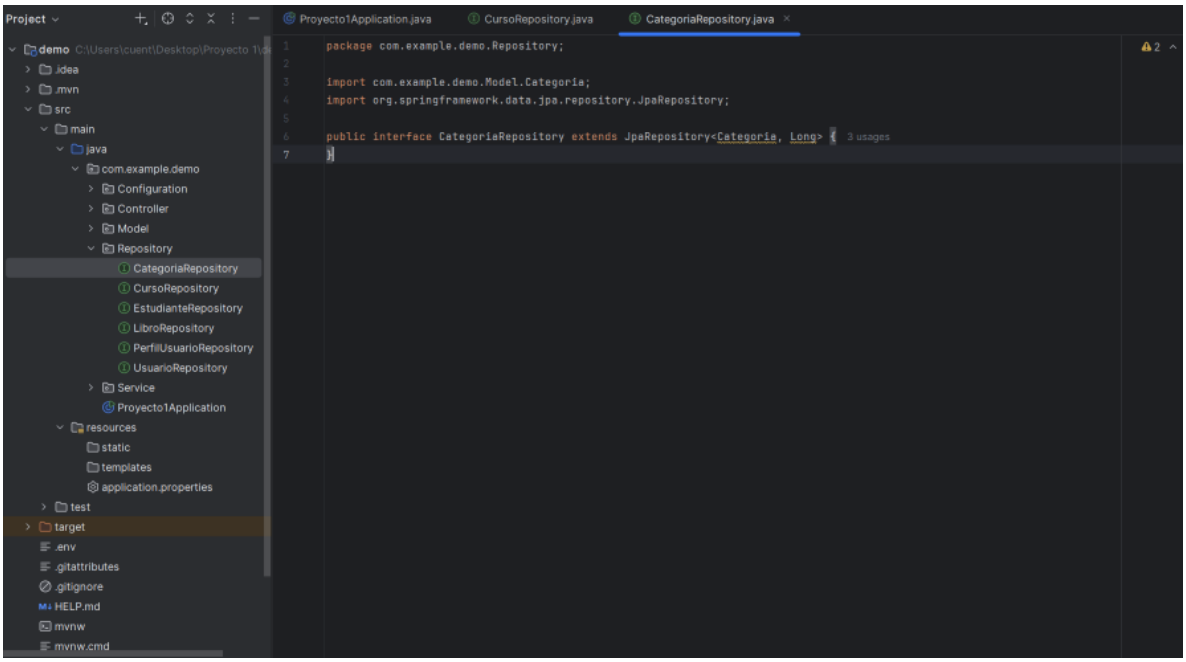


Imagen: Package Repository

Model:

El package model contiene las clases que representan las entidades del sistema, las cuales están mapeadas a las tablas de la base de datos mediante anotaciones de JPA. Se incluyen entidades como Usuario, PerfilUsuario, Curso, Libro, Estudiante y Categoria, donde cada clase define los atributos y las relaciones entre ellas. Estas entidades permiten estructurar la información que se almacena en PostgreSQL y sirven como base para las operaciones realizadas en las demás capas del sistema.

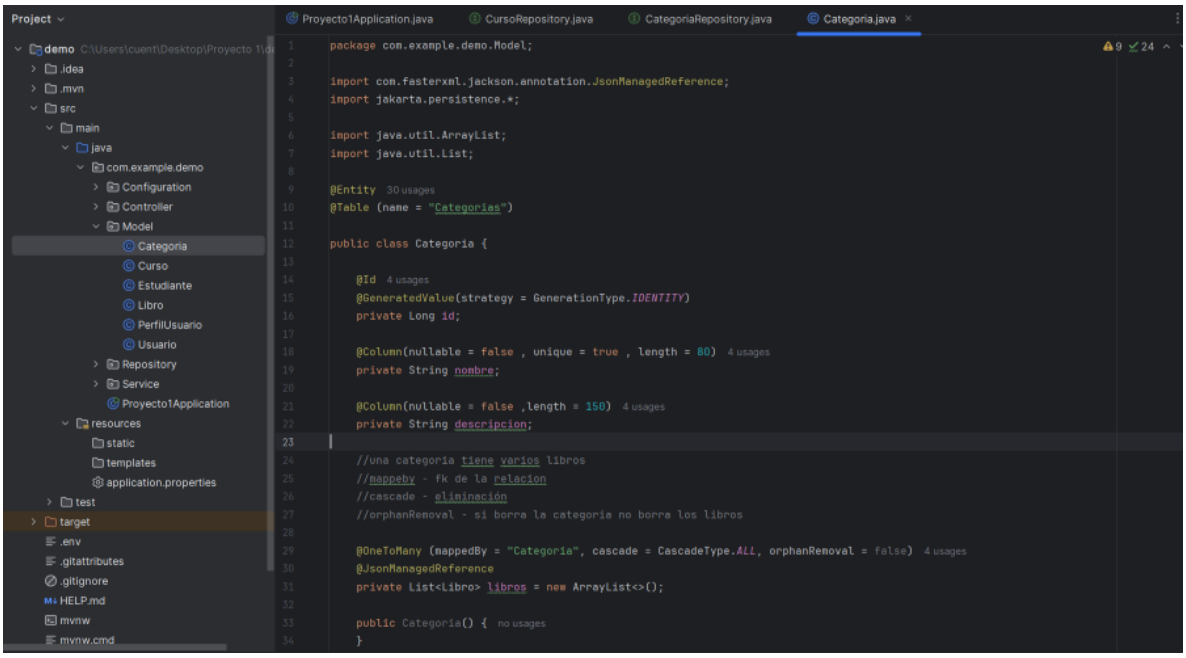


Imagen: Package Model

Configuration:

El package configuration se utilizó para definir la configuración de Swagger OpenAPI, permitiendo la documentación de la API. A través de la clase SwaggerConfig, se establece información general como el nombre, la versión y la descripción del sistema, lo que facilita la visualización y prueba de los endpoints desde el navegador. Esta configuración permite tener una documentación clara y organizada de los servicios desarrollados en la API.

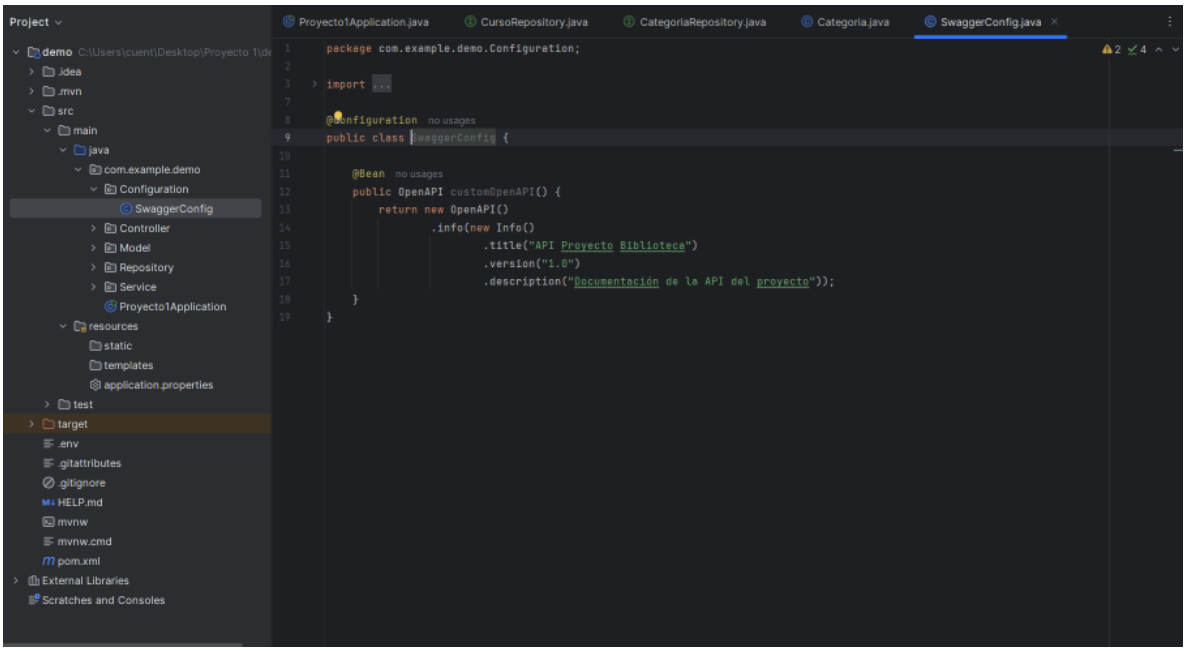


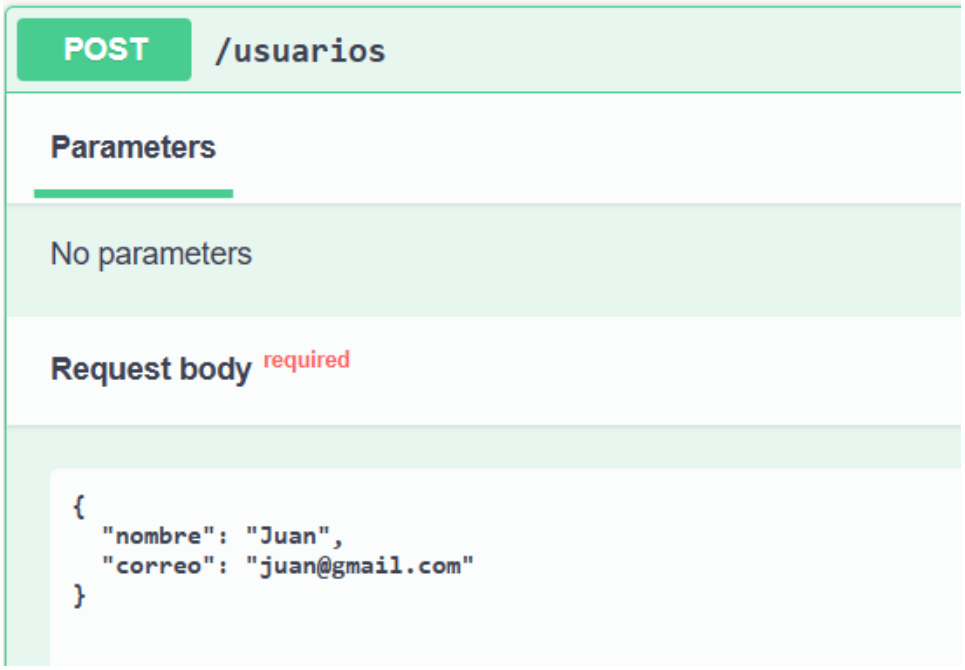
Imagen: Package Configuration.

Relación entre JPA y SQL.

En este proyecto se utilizó JPA como herramienta para la persistencia de datos, por lo cual no fue necesario realizar consultas SQL de forma manual, ya que estas son generadas automáticamente por Hibernate a partir de las operaciones definidas en el código. A través de los repositorios que extienden JpaRepository, se implementaron las operaciones CRUD para cada una de las entidades del sistema. A continuación, se presentan estas operaciones junto con sus equivalentes en SQL, con el fin de evidenciar la relación entre las acciones realizadas en JPA y las consultas ejecutadas en la base de datos.

Tabla Usuarios:

Peticion – Post:



JPA:

```
@Override
public Estudiante guardar(Estudiante estudiante) {
    return repository.save(estudiante);
}
```

Equivalente en SQL:

```
INSERT INTO usuarios (nombre, correo)
VALUES ('Juan', 'juan@gmail.com');
```

Respuesta:

201
Undocumented

Response body

```
{
  "id": 1,
  "nombre": "Juan",
  "correo": "juan@gmail.com",
  "perfil": null
}
```

Petición - Get:

GET /usuarios

Parameters

No parameters

JPA:

```java
@Override
public List<Estudiante> listar() {
    return repository.findAll();
}
```

Equivalente en SQL:

```sql
SELECT * FROM usuarios;
```

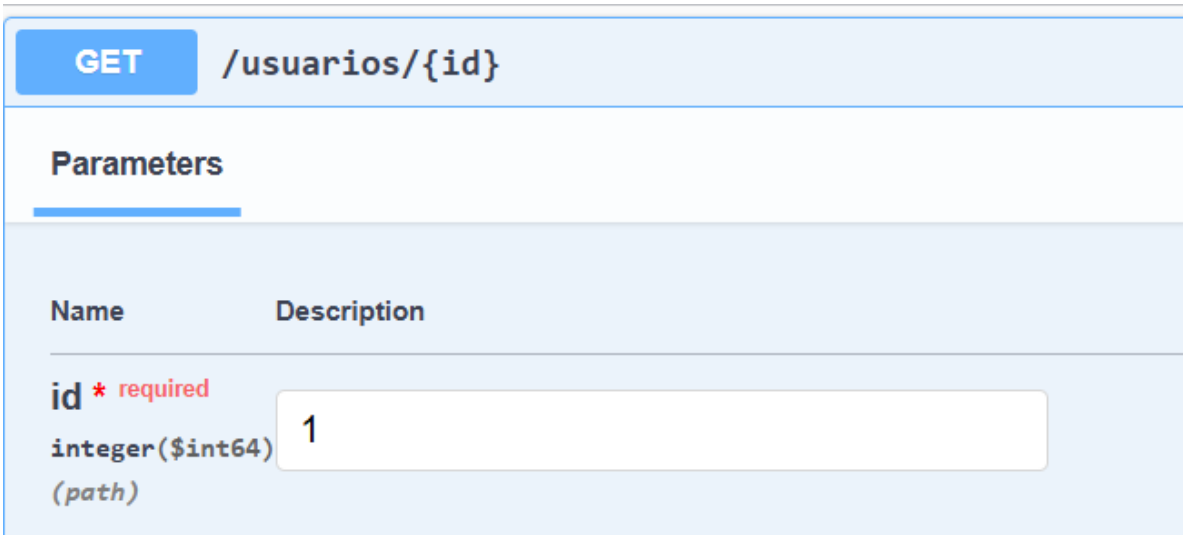
Respuesta:

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "nombre": "Juan",
    "correo": "juan@gmail.com",
    "perfil": null
  }
]
```

Petición Get-Id:



JPA:

```
@Override
public Estudiante buscarPorId(Long id) {
    return repository.findById(id).orElse( other: null);
}
```

Equivalente en SQL:

```
SELECT * FROM usuarios
WHERE id = 1;
```

Respuesta:

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "nombre": "Juan",
  "correo": "juan@gmail.com",
  "perfil": null
}
```

Petición - Put:

PUT /usuarios/{id}

Parameters

Name	Description
id * required integer($int64) (path)	1

Request body required

```
{
  "nombre": "Juan Luis",
  "correo": "juan@gmail.com"
}
```

JPA:

```
@Override
public Estudiante actualizar(Long id, Estudiante estudiante) {
    estudiante.setId(id);
    return repository.save(estudiante);
}
```

Equivalente SQL:

```sql
UPDATE usuarios
SET nombre = 'Juan Luis', correo = 'juan@gmail.com'
WHERE id = 1;
```

Respuesta:

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "nombre": "Juan Luis",
  "correo": "juan@gmail.com",
  "perfil": null
}
```

Petición – Delete:

DELETE /usuarios/{id}

Parameters

Name	Description
id * required integer($int64) (path)	1

JPA:

```
@Override
public void eliminar(Long id) {
    repository.deleteById(id);
}
```

Equivalente SQL:

```
DELETE FROM usuarios
WHERE id = 1;
```

Respuesta:

Server response

Code	Details
204 *Undocumented*	**Response headers**

```
connection: keep-alive
date: Wed,18 Mar 2026 11:08:13 GMT
keep-alive: timeout=60
```

Las demás entidades del proyecto siguen la misma lógica de implementación del CRUD, ya que todas están construidas bajo la misma arquitectura por capas utilizando Controller, Service y Repository. Esto hace que la estructura del código sea uniforme, permitiendo que operaciones como crear, consultar, actualizar y eliminar se realicen de manera similar en cada caso a través de métodos proporcionados por JPA.

La principal diferencia entre ellas radica en los atributos propios de cada entidad y en las relaciones que algunas manejan (como uno a uno o uno a muchos), lo cual puede influir

ligeramente en la forma en que se envían o reciben los datos, pero no cambia la base del funcionamiento del CRUD dentro del sistema.

Entidades en Swagger (Open/Api):

usuario-controller

GET /usuarios/{id}

PUT /usuarios/{id}

DELETE /usuarios/{id}

GET /usuarios

POST /usuarios

perfil-usuario-controller

GET /perfiles/{id}

PUT /perfiles/{id}

DELETE /perfiles/{id}

GET /perfiles

POST /perfiles

libro-controller

GET /libros/{id}

PUT /libros/{id}

DELETE /libros/{id}

GET /libros

POST /libros

estudiante-controller

GET /estudiantes/{id}

PUT /estudiantes/{id}

DELETE /estudiantes/{id}

GET /estudiantes

POST /estudiantes

curso-controller

- GET /cursos/{id}
- PUT /cursos/{id}
- DELETE /cursos/{id}
- GET /cursos
- POST /cursos

categoria-controller

- GET /categorias/{id}
- PUT /categorias/{id}
- DELETE /categorias/{id}
- GET /categorias
- POST /categorias

Verificación de registro en la base de datos:

Para validar el correcto funcionamiento del sistema, se verificó que las operaciones realizadas desde la API se reflejaran adecuadamente en la base de datos. En este caso, se utilizó Supabase como gestor de la base de datos PostgreSQL, lo que permitió observar directamente las tablas y sus registros.

De esta manera, se evidenció que la información se almacena y se gestiona correctamente, confirmando un funcionamiento adecuado de la integración entre JPA, Hibernate y la base de datos.

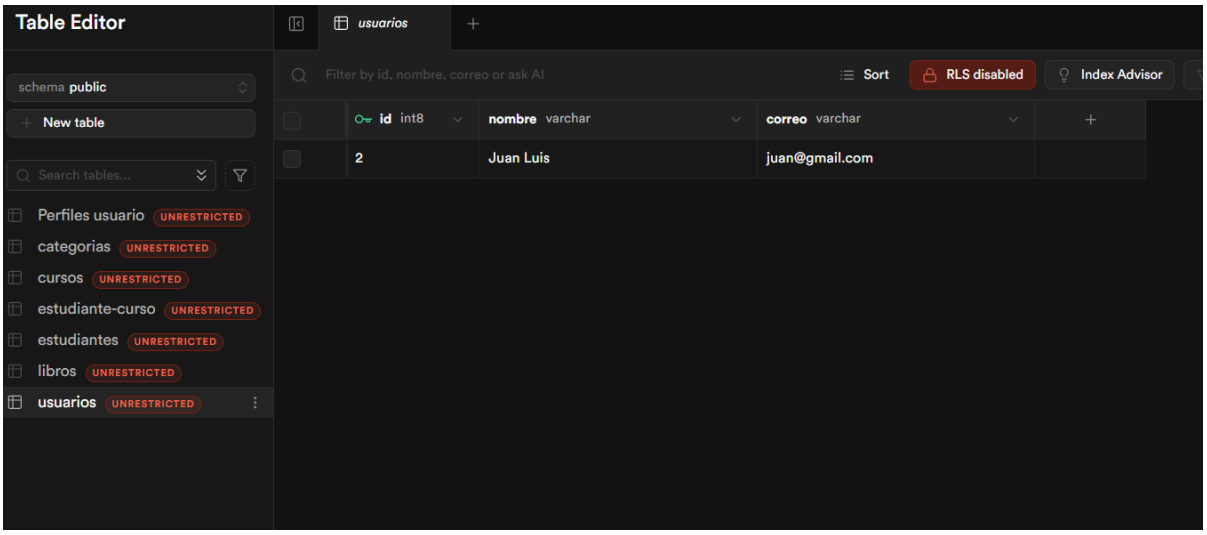


Imagen: Tabla de Usuarios Base de Datos

Conclusiones:

- El desarrollo del proyecto permitió comprender de manera más clara y práctica el funcionamiento de la arquitectura por capas en Spring Boot, destacando la importancia de separar responsabilidades entre los paquetes controller, service y repository. Esta organización no solo facilita la lectura del código, sino que también mejora su mantenimiento y permite escalar el sistema de manera más ordenada.
- Se evidenció la utilidad de JPA con Hibernate en la interacción con la base de datos PostgreSQL, ya que permite gestionar las entidades como objetos dentro del código, evitando la necesidad de escribir consultas SQL manuales. Esto simplifica el desarrollo, reduce errores y hace más eficiente la implementación de las operaciones CRUD.
- A través de las pruebas realizadas, se comprobó que las operaciones CRUD (crear, leer, actualizar y eliminar) funcionan correctamente en cada una de las entidades del sistema, lo que demuestra una adecuada integración entre la API, JPA y la base de datos, garantizando así un manejo correcto y consistente de la información.

Link de Git Hub:

https://github.com/JuanJGr/Api_Bilblioteca.git